Lab -2 – Task:

Reg.No : AP25110010807

Name : S.Jignesh

Question 1:

Algorithm 1:

Code :

main.cpp

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int x, n;

    cout << "Enter x: ";
    cin >> x;

    cout << "Enter n: ";
    cin >> n;

    int sum = 0;

    if (x == 1) {
        sum = n + 1;
    }
    else {
        sum = (pow(x, n) - 1) / (x - 1);
    }

    cout << "Sum = " << sum;

    return 0;
}
```

Output:

```
Enter x: 2
Enter n: 3
Sum = 7

...Program finished with exit code 0
Press ENTER to exit console.
```

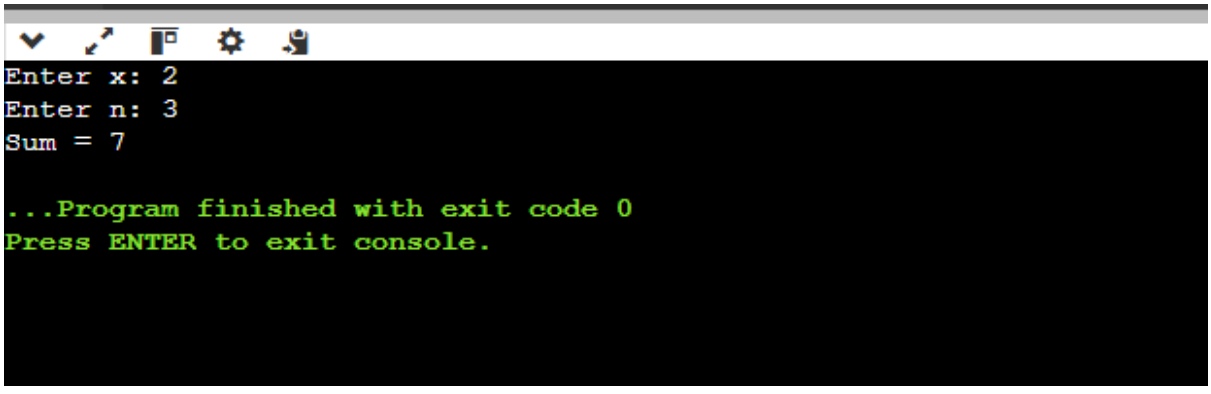
Algorithm 2:

Code:

main.cpp

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int x, n;
    int sum = 0;

    cout << "Enter x: ";
    cin >> x;

    cout << "Enter n: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        sum = sum + pow(x, i);
    }

    cout << "Sum = " << sum;

    return 0;
}
```

Output:



Question 2:

Algorithm 1:

Code:

main.cpp

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int n, k;

    cout << "Enter size of array: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Enter k: ";
    cin >> k;

    sort(arr, arr + n);

    cout << "Kth smallest element = " << arr[k - 1];

    return 0;
}
```

Output:

```
Enter size of array: 5
Enter elements: 1
2
3
4
5
Enter k: 2
Kth smallest element = 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Algorithm 2:

Code:

main.cpp

```cpp
#include <iostream>
#include <queue>
using namespace std;

int main() {
    int n, k;

    cout << "Enter size of array: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Enter k: ";
    cin >> k;

    priority_queue<int> maxHeap;

    for (int i = 0; i < k; i++) {
        maxHeap.push(arr[i]);
    }

    for (int i = k; i < n; i++) {

        if (arr[i] < maxHeap.top()) {
            maxHeap.pop();
            maxHeap.push(arr[i]);
        }
    }

    cout << "Kth smallest element = " << maxHeap.top();

    return 0;
}
```

Output:

```
Enter size of array: 5
Enter elements: 1
2
3
4
5
Enter k: 2
Kth smallest element = 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 3:

Code:

main.cpp

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = NULL;
        right = NULL;
    }
};

Node* insert(Node* root, int value) {

    if (root == NULL) {
        return new Node(value);
    }

    if (value < root->data) {
        root->left = insert(root->left, value);
    }
    else if (value > root->data) {
        root->right = insert(root->right, value);
    }

    return root;
}

Node* findMin(Node* root) {

    if (root->left == NULL) {
        return root;
    }

    return findMin(root->left);
}

```

main.cpp

```cpp

Node* deleteNode(Node* root, int value) {

    if (root == NULL) {
        return root;
    }

    if (value < root->data) {
        root->left = deleteNode(root->left, value);
    }
    else if (value > root->data) {
        root->right = deleteNode(root->right, value);
    }
    else {

        if (root->left == NULL) {
            Node* temp = root->right;
            delete root;
            return temp;
        }

        if (root->right == NULL) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        Node* temp = findMin(root->right);

        root->data = temp->data;

        root->right = deleteNode(root->right, temp->data);
    }

    return root;
}

```

main.cpp

```cpp
void inorder(Node* root) {

    if (root == NULL) {
        return;
    }

    inorder(root->left);

    cout << root->data << " ";

    inorder(root->right);
}

int main() {

    Node* root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    cout << "BST after insertion: ";
    inorder(root);

    root = deleteNode(root, 30);

    cout << "\nBST after deleting 30: ";
    inorder(root);

    root = deleteNode(root, 70);

    cout << "\nBST after deleting 70: ";
    inorder(root);

    return 0;
}
```

Output:

```
BST after insertion: 20 30 40 50 60 70 80
BST after deleting 30: 20 40 50 60 70 80
BST after deleting 70: 20 40 50 60 80

...Program finished with exit code 0
Press ENTER to exit console.
```